

The Claude Agent SDK — Building On the Loop Instead of Rebuilding It

Track 16.7 —  Claude Code and the Agent SDK · 16-ai-tech-stack/tracks/claude-code/notes/02-agent-sdk-and-building-on-it.md

1. What the SDK actually is

"The Agent SDK gives you the same tools, agent loop, and context management that power Claude Code, programmable in Python and TypeScript." — [Agent SDK overview](#)

That is the whole pitch, and it is worth taking literally. When you `pip install claude-agent-sdk`, you are not installing a thin wrapper around the Messages API. You are installing **Claude Code as a library**: the SDK ships a native Claude Code binary as a platform dependency and drives it as a subprocess. Your Python or TypeScript process is the *host*; the agent loop runs in a child process and streams messages back.

Consequences that surprise people:

- Your agent gets `Read`, `Write`, `Edit`, `Bash`, `Grep`, `Glob`, `WebSearch`, `WebFetch`, `Agent` and the rest **for free**, already implemented, already permission-gated.
- It reads `.claude/` on disk — skills, commands, subagents, settings — the same way the CLI does, unless you turn that off.
- It ships hooks, subagents, MCP, sessions, plugins and permission modes as first-class options.
- The failure modes are Claude Code's failure modes, and the docs for them are the same docs.

The name changed: `@anthropic-ai/claude-code` → `@anthropic-ai/claude-agent-sdk`, and `claude-code-sdk` → `claude-agent-sdk` (imports: `claude_code_sdk` → `claude_agent_sdk`, `ClaudeCodeOptions` → `ClaudeAgentOptions`). If you find a tutorial using the old names, it is stale.

```
# Python
python3 -m venv .venv && source .venv/bin/activate
pip install claude-agent-sdk          # or: uv add claude-agent-sdk

# TypeScript
npm install @anthropic-ai/claude-agent-sdk
npm install --save-dev tsx
```

Requirements: Python 3.10+ or Node 18+, and an `ANTHROPIC_API_KEY` in the environment of the process that runs your agent. The SDK does **not** read `.env` files for you. Bedrock / Claude Platform on AWS / Google Cloud Agent Platform / Microsoft Foundry are supported via `CLAUDE_CODE_USE_BEDROCK=1`, `CLAUDE_CODE_USE_ANTHROPIC_AWS=1`, `CLAUDE_CODE_USE_VERTEX=1`, `CLAUDE_CODE_USE_FOUNDRY=1`.

Licensing note, because it catches people: unless previously approved, Anthropic does not allow third-party developers to offer claude.ai login or subscription rate limits for products built on the Agent SDK. Ship with API-key auth. There are also branding rules — "Claude Agent" and "Powered by Claude" are allowed; "Claude Code" is not.

2. What it gives you over raw API calls

If you build an agent on the Messages API you will write, debug and maintain all of this yourself:

You would have to build	The SDK ships it
The tool-use loop: send → parse <code>tool_use</code> → execute → send <code>tool_result</code> → repeat until stop	<code>query()</code> / <code>ClaudeSDKClient</code> , streaming <code>AsyncIterator[Message]</code>
File read/write/edit with diffing, line limits, encoding handling	<code>Read</code> , <code>Write</code> , <code>Edit</code> built in
Shell execution with timeouts, backgrounding, output truncation	<code>Bash</code> built in
Search that doesn't blow the window	<code>Grep</code> , <code>Glob</code> , <code>LSP</code> built in
Compaction when the window fills	automatic, plus <code>PreCompact</code> / <code>PostCompact</code> hooks
A permission layer with modes and rules	<code>permission_mode</code> , <code>allowed_tools</code> , <code>disallowed_tools</code> , <code>can_use_tool</code>
Sub-agents with isolated context windows	<code>agents={...}</code> / <code>.claude/agents/</code> , plus built-in <code>Explore/Plan</code>
Interception points before and after every tool call	<code>hooks={HookEvent: [HookMatcher(...)]}</code>
Session persistence, resume, fork	<code>continue_conversation</code> , <code>resume</code> , <code>fork_session</code> , <code>session_store</code>
External tool integration	MCP, in-process and out
Cost/usage accounting per model	<code>ResultMessage.total_cost_usd</code> , <code>.model_usage</code>
Prompt caching strategy	managed for you

The honest framing: **the agent loop is not the hard part — the hundred details around it are.** Truncating a 50MB log so it doesn't eat the window, deciding when a `cd` persists, making `Edit` fail loudly instead of silently writing the wrong region, knowing that a tool result should be saved to a file past 500k characters. That is the part you are buying.

3. The minimum viable agent

```
import asyncio
from claude_agent_sdk import query, ClaudeAgentOptions, AssistantMessage, ResultMessage

async def main():
    async for message in query(
        prompt="Review utils.py for crash bugs and fix them.",
        options=ClaudeAgentOptions(
            allowed_tools=["Read", "Edit", "Glob"],
            permission_mode="acceptEdits",
        ),
    ):
        if isinstance(message, AssistantMessage):
            for block in message.content:
                if hasattr(block, "text"):
                    print(block.text)
        elif isinstance(message, ResultMessage):
            print(f"Done: {message.subtype} ${message.total_cost_usd}")

asyncio.run(main())
```

```
import { query } from "@anthropic-ai/claude-agent-sdk";

for await (const message of query({
  prompt: "Review utils.py for crash bugs and fix them.",
  options: { allowedTools: ["Read", "Edit", "Glob"], permissionMode: "acceptEdits" },
})) {
  if (message.type === "result") console.log(message.subtype);
}
```

`query()` starts a fresh session each call and returns an async iterator of messages. `ClaudeSDKClient` (Python) reuses one session across exchanges, supports interrupts, and is what you want behind a chat UI.

	<code>query()</code>	<code>ClaudeSDKClient</code>
Session	new each call	reused
Conversation	single exchange	multi-turn in one context
Interrupts	no	yes
Hooks / custom tools	yes	yes
Use for	one-off tasks, CI, batch	chat interfaces, interactive apps

The message types you will actually match on: `AssistantMessage` (content blocks), `UserMessage`, `SystemMessage`, `StreamEvent` (with `include_partial_messages=True`), and `ResultMessage` — the terminal one, carrying `subtype` (`success`, `error_during_execution`, `error_max_turns`, `error_max_budget_usd`, ...), `result`, `total_cost_usd`, `usage`, `model_usage`, `num_turns`, `permission_denials`, `terminal_reason`.

4. Python vs TypeScript

Same loop, same options, different spelling. The practical differences:

	Python (claude-agent-sdk)	TypeScript (@anthropic-ai/claude-agent-sdk)
Naming	snake_case options (allowed_tools, permission_mode)	camelCase (allowedTools, permissionMode)
Custom tool schema	dict of types ({"a": float}) or JSON Schema	Zod schema, handler args typed from it
Entry points	query(), ClaudeSDKClient	query(), startup() (pre-warms the subprocess)
Structured output	Pydantic / JSON Schema	Zod / JSON Schema
Gotcha	AgentDefinition fields are camelCase even in Python — they map to the shared wire format	bundled binary arrives via npm optional deps: npm ci --omit=optional leaves you without one

Pick TypeScript if the agent lives inside a Node service or you want the Zod typing. Pick Python if the agent sits next to your data/ML code. There is no capability gap worth choosing on. If you need a third language, the docs are explicit: run the CLI as a subprocess with `-p` and `--output-format json`.

5. Custom tools: the in-process MCP server

Custom tools in the Agent SDK are MCP tools served from inside your own process — no subprocess, no socket.

```
from typing import Any
from claude_agent_sdk import tool, create_sdk_mcp_server, ClaudeAgentOptions

@tool("lookup_order", "Look up an order by ID in the internal DB", {"order_id": str})
async def lookup_order(args: dict[str, Any]) -> dict[str, Any]:
    row = await db.fetch_one("SELECT * FROM orders WHERE id = $1", args["order_id"])
    if row is None:
        return {"content": [{"type": "text", "text": "No such order."}], "isError": True}
    return {
        "content": [{"type": "text", "text": f"Order {row['id']}: {row['status']}"}],
        "structuredContent": dict(row),          # machine-readable twin of the text
    }

orders = create_sdk_mcp_server(name="orders", version="1.0.0", tools=[lookup_order])

options = ClaudeAgentOptions(
    mcp_servers={"orders": orders},
    allowed_tools=["mcp__orders__lookup_order"],  # mcp__<server-key>__<tool-name>
)
```

Four parts to a tool: **name**, **description** (this is what Claude reads to decide when to call it — it is a prompt, write it like one), **input schema**, **handler**. The handler returns `content` (an array of blocks typed `text`, `image`, `audio`, `resource`, `resource_link`), optionally `structuredContent`, optionally `isError: true` so you control the message Claude reads instead of leaking a raw exception.

Details that matter in production:

- The key you use in `mcp_servers` becomes the `{server_name}` segment of the tool's fully-qualified name. Rename the key, break your `allowed_tools`.
- `ToolAnnotations(readOnlyHint=True)` lets Claude call the tool **in parallel** with others. Set it honestly.
- `maxResultSizeChars` controls how much of a text result stays inline before Claude Code spills it to a file.
- Python's dict schema treats every key as required. To make a parameter optional, leave it out of the schema, mention it in the description, and read it with `args.get(...)`. TypeScript uses `.default()` on the Zod field.
- Past a few dozen tools, turn on [tool search](#) so only the relevant ones load.

External MCP servers (stdio, HTTP, SSE) go in the same `mcp_servers` dict as config objects. See `../09-mcp/` for the protocol itself.

6. Controlling permissions programmatically

The SDK evaluates permissions in a **fixed documented order**, and knowing it prevents most "why did my callback never fire" bugs:

- | | |
|------------------------------|--|
| 1. Hooks | → can deny outright; an "allow" here does NOT skip steps 2–3 |
| 2. Deny rules | → <code>disallowed_tools</code> + <code>settings.json</code> ; wins even in <code>bypassPermissions</code> |
| 3. Ask rules | → from <code>settings.json</code> ; falls through to <code>can_use_tool</code> |
| 4. Permission mode | → <code>bypassPermissions</code> / <code>acceptEdits</code> / <code>plan</code> behave per mode |
| 5. Allow rules | → <code>allowed_tools</code> + <code>settings.json</code> ; approved here, done |
| 6. <code>can_use_tool</code> | → your callback. Skipped (denied) in <code>dontAsk</code> mode |

The trap, stated by the docs: *"Auto-approved tools never reach `canUseTool`."* If you pass a bare `allowed_tools=["Read"]` and also a `can_use_tool` callback expecting to audit every read, your callback never runs. The TypeScript SDK emits a Node process warning when it detects this shape. To gate **every** call regardless of mode and rules, use a `PreToolUse` hook, not the callback.

The callback:

```
from claude_agent_sdk import (
    ClaudeAgentOptions, PermissionResultAllow, PermissionResultDeny, ToolPermissionContext,
)

async def gate(tool_name: str, input_data: dict, ctx: ToolPermissionContext):
    if tool_name == "Bash" and "rm -rf" in input_data.get("command", ""):
        return PermissionResultDeny(message="Destructive command blocked by policy.")
    if tool_name == "Write" and input_data.get("file_path", "").endswith(".env"):
        return PermissionResultDeny(message="Refusing to write secrets.", interrupt=True)
    return PermissionResultAllow()  # or PermissionResultAllow(updated_input={...})

options = ClaudeAgentOptions(
    tools=["Read", "Grep", "Glob", "Bash", "Write"],
    can_use_tool=gate,
    permission_mode="default",
)
```

Modes: `default`, `acceptEdits`, `plan`, `dontAsk`, `bypassPermissions`, `auto`. The locked-down recipe the docs recommend is `allowed_tools=[...]` plus `permission_mode="dontAsk"`: listed tools run, everything else that would have prompted is denied. For an unattended agent that is the right default.

Two more controls worth knowing:

- `disallowed_tools=["Bash"]` removes the tool definition from the request entirely — Claude never sees it and cannot try. `disallowed_tools=["Bash(rm *)"]` keeps Bash but denies matching calls in every mode.
- `setting_sources` decides whether the SDK reads `~/.claude/settings.json`, `.claude/settings.json`, `.claude/settings.local.json`. Pass `setting_sources=[]` for a hermetic agent that depends on nothing on disk — and note that this also means **no CLAUDE.md**: to load project instructions you must include `"project"`. In a multi-tenant service, `[]` is almost always what you want.

Hooks in the SDK are Python/TS callbacks rather than shell commands, with the same events and the same `hookSpecificOutput` decision shape:

```
from claude_agent_sdk import HookMatcher

async def block_env(input_data, tool_use_id, context):
    if input_data["tool_input"].get("file_path", "").endswith(".env"):
        return {"hookSpecificOutput": {
            "hookEventName": input_data["hook_event_name"],
            "permissionDecision": "deny",
            "permissionDecisionReason": "Cannot modify .env files",
        }}
    return {}

options = ClaudeAgentOptions(
    hooks={"PreToolUse": [HookMatcher(matcher="Write|Edit", hooks=[block_env])]}
)
```

7. Streaming, sessions, structured output

Streaming input vs single message. Passing a `str` prompt is single-message mode. Passing an `AsyncIterable` of message dicts is streaming input mode — the shape you need for a live chat, for sending images mid-conversation, or for queueing user turns while the agent works.

Streaming output is the default: `query()` yields messages as they arrive, so you can render tool calls and partial reasoning live. `include_partial_messages=True` adds token-level `StreamEvent`s. For a background job you can simply collect the iterator and read the final `ResultMessage`.

Sessions. `continue_conversation=True` picks up the most recent; `resume="<session-id>"` reopens a specific one; `fork_session=True` branches instead of appending. `session_store` mirrors transcripts to S3/Redis/your backend so another host can resume them — the piece you need for a horizontally-scaled service.

Structured output. `output_format={"type": "json_schema", "schema": {...}}` gives you validated JSON *after* the agent finishes its multi-turn tool use, surfaced on `ResultMessage.structured_output`. This is the

clean way to get a typed result out of an agentic workflow instead of regexing the final message. The CLI equivalent is `--json-schema` in print mode.

Cost. Read `ResultMessage.total_cost_usd` and `.model_usage`. Careful: `usage` covers the main loop only — subagent tokens are **not** in it; `model_usage` covers everything including subagents and compaction. In streaming input mode both are cumulative across turns, so read the latest result rather than summing. `max_budget_usd` is your circuit breaker.

8. SDK vs Messages API vs LangGraph — an honest decision table

Dimension	Messages API (anthropic client)	Claude Agent SDK	LangGraph
What it is	HTTP API + thin client; you own the loop	Claude Code as a library; agent loop included	A graph/state-machine framework you compose yourself
Tools	you define schemas and execute every call	~30 built-ins + your custom MCP tools	you define and execute; rich ecosystem
Control flow	whatever you write	model-driven, turn by turn	explicit graph — nodes, edges, conditionals
Determinism	total (it's your code)	low: Claude decides the next step	high: the graph decides; the model fills nodes
Filesystem/shell access	none unless you build it	first-class, permission-gated	none unless you build it
Permissions	none — you are the enforcement	modes + allow/ask/deny + hooks + callback	none built in
Context management	you compact	automatic compaction, subagent isolation	you manage state; checkpoints persist it
Model lock-in	Anthropic	Anthropic	provider-agnostic
Human-in-the-loop	you build it	<code>can_use_tool</code> , <code>AskUserQuestion</code>	interrupts + checkpoints, designed for it
Overhead	none	spawns a Claude Code subprocess per session	a Python/JS dependency
Best at	narrow, high-volume, latency-sensitive calls	software/ops tasks on a real filesystem	multi-step business workflows needing auditable paths

Choose the Messages API when the job is one bounded model call — classify, extract, summarize, rewrite — or a tool loop with two or three of *your* tools and no filesystem. Don't spawn an agent harness to call one function. See `../..../04-llm/` and the `claude-api` reference.

Choose the Agent SDK when the work looks like software engineering or operations on a real machine: read a repo, edit files, run a build, triage logs, migrate a codebase, review a PR, automate a runbook. You want the built-in tools, and you want permissions you can defend in a review. It is also the fastest path from a Claude

Code workflow that already works to a scheduled/hosted version of it — the Claude Code GitHub Action is itself built on the SDK.

Choose LangGraph when the control flow is a business requirement rather than a model decision: fixed stages, conditional routing, retries with explicit compensation, human approval gates in defined places, an audit trail of which path ran. Also when you must stay provider-agnostic. See `../langgraph/`.

They compose. A LangGraph node can call an Agent SDK agent for the "go fix the repo" step while the graph owns the surrounding workflow. That is usually a better answer than forcing either one to do the other's job.

The failure mode to avoid: **using an agentic loop where a state machine was the requirement.** If a compliance reviewer will ask "which steps ran, in what order, and who approved step 4", a model choosing its next action turn by turn is the wrong architecture, no matter how good the model is. The inverse failure — building a 40-node graph to do "read the repo and fix the type errors" — is just as common and wastes a month.

9. Deployment realities

- **Subprocess architecture.** Each session spawns a Claude Code process. Budget memory and file descriptors accordingly; the [hosting guide](#) covers Docker, Kubernetes, multi-tenant isolation and sandbox providers.
 - **Isolate the filesystem.** An agent with Bash is an agent with your machine. Containers, per-tenant working directories, and `setting_sources=[]` so a tenant's repo can't inject hooks into your host process.
 - **The lethal trifecta applies here too, harder.** Your service reads untrusted input (user-supplied repos, tickets, web pages), holds credentials, and can act. Everything in `01-claude-code-core-concepts.md §5` [16-ai-tech-stack/tracks/claude-code/notes/01-claude-code-core-concepts.md] and `../13-ai-security/` applies, with the extra wrinkle that there is no human at a permission prompt to catch the weird one. Your `PreToolUse` hook is the human.
 - **Observability.** OpenTelemetry export is supported; `ResultMessage` gives you per-run cost; `permission_denials` tells you what your policy actually blocked — read it, it's the highest-signal log line you have.
 - **Timeouts.** `API_TIMEOUT_MS`, `CLAUDE_CODE_MAX_RETRIES`, and the stall watchdog are configured through `ClaudeAgentOptions.env`. Worst-case wall time is roughly `API_TIMEOUT_MS × (retries + 1)` — check the arithmetic before you set a generous timeout and a generous retry count together.
-

10. Where this returns

Idea here	Where it returns
Tool loops, ReAct, planning as theory	../../../../06-ai-agents/
Multi-agent orchestration and hand-offs	../../../../07-agentic-ai/
In-process MCP servers, transports, protocol design	../../../../09-mcp/
Explicit graphs, checkpoints, human-in-the-loop	../langgraph/
Agent-specific threat modelling	../../../../13-ai-security/
Evaluating an agent you just built	../../../../15-ai-evals/
Async, typing and packaging the host process	../python/